



INGENIERÍA EN SISTEMAS COMPUTACIONALES

Sistemas Programables

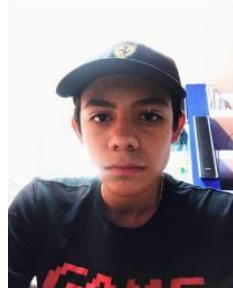
Módulo 5: Proyecto Final

Reporte Final, Repositorio y Video (Integración Total)

Agente Robótico Autónomo "GARRA-OS"

P R E S E N T A:

**Alcalá Ramos Luz Estefanía
Bahena Mora Emilio Salvador
Casas Bastidas José Iván
Fischer González Patrick**



DOCENTE:

Ma. Verónica Tapia Ibarra

LEÓN, GUANAJUATO.

2026



Índice

Índice	2
1. Objetivo general.....	4
1.1 Objetivos específicos.....	4
1.2 Evolución respecto a la propuesta inicial	4
2. Materiales y equipos	6
2.1 Conexionado por nodo (pinout)	6
Nodo Cerebro (ESP32-S3)	6
Nodo Ejecución (ESP32 DevKit V1).....	7
Nodo Expresión (ESP32 DevKit V1)	7
3. Marco teórico	8
3.1 Microcontroladores ESP32 y ESP32-S3.....	8
3.2 Capa de Abstracción de Hardware (HAL).....	8
3.3 PWM, ESC y lectura de receptores RC	8
3.4 Comunicación entre dispositivos: ESP-NOW y UART (en lugar de MQTT)	8
3.5 Persistencia en la nube y tiempo real: Supabase (en lugar de Firebase).....	8
3.6 Autonomía reactiva embebida (en lugar de visión por computadora).....	9
4. Diagrama del sistema	10
4.1 Arquitectura de datos (Nube / Autonomía).....	11
4.2 Descripción de los bloques.....	11
5. Roles del equipo	12
6. Desarrollo	13
6.1 Pilar 1 — Biblioteca de Control de Hardware (HAL).....	13
Control de motores	13
Proyección de expresiones	13
Sensor de caída.....	13
6.2 Pilar 2 — Integración total de la malla de comunicación (ESP-NOW ↔ UART).....	14
Secuencia de arranque (handshake / “POST”)	14
6.3 Pilar 3 — Autonomía reactiva: el reflejo de emergencia (en lugar de IA de visión)	15
Jerarquía de prioridades	15
Máquinas de estado por nodo.....	15
6.4 Pilar 4 — Persistencia en la nube y dashboard (Supabase ↔ ESP32 ↔ Interfaz).....	15
6.5 Desglose del protocolo del ventilador holográfico	17
6.6 Generación de la coreografía (herramienta Grabadora)	17
6.7 Evolución del firmware por versiones	18
7. Retos técnicos y soluciones	19

8. Resultados.....	19
9. Conclusiones Grupales	21
10. Análisis individual por integrante	22
11. Lista de verificación de entrega.....	23
12. Glosario técnico	23
13. Referencias.....	24

1. Objetivo general

Diseñar, construir y poner en marcha GARRA-OS: el sistema de control distribuido del robot de combate Chappie que integra, como un producto único e indivisible, tres pilares: el dispositivo físico terminado, una capa de software profesional organizada y una infraestructura de datos en la nube. El sistema debe permitir que el robot sea comandado de forma remota desde una interfaz web, conserve la prioridad del piloto humano en todo momento, reaccione de manera autónoma ante condiciones de peligro y reporte su estado en tiempo real, demostrando dominio práctico de microcontroladores, comunicación entre dispositivos, electrónica de potencia y arquitectura de software por capas.

1.1 Objetivos específicos

- Implementar una Capa de Abstracción de Hardware (HAL) por nodo, de modo que la lógica de alto nivel nunca manipule pines o temporizadores directamente.
- Establecer una malla de comunicación de baja latencia entre tres microcontroladores ESP32 usando ESP-NOW (radio) y UART (cable), sin depender de un broker ni de Internet en el lazo crítico de control.
- Integrar una capa de persistencia y mando en la nube con Supabase (PostgreSQL + REST + Realtime), separando el flujo de comandos (Web → robot) del flujo de telemetría (robot → Web).
- Construir un sistema de seguridad jerárquico cuyo reflejo de emergencia, disparado por sensor, tenga prioridad absoluta sobre cualquier orden manual o remota.
- Desarrollar una interfaz web de mando y monitoreo que muestre la telemetría de los tres nodos en tiempo real.
- Encapsular el prototipo en un chasis con apariencia de producto terminado y documentar el repositorio de forma profesional.

1.2 Evolución respecto a la propuesta inicial

El proyecto cambió de manera sustancial respecto a la propuesta con la que se arrancó el semestre. La concepción original era un agente “social” con sensores de proximidad (HC-SR04), unidad inercial (MPU6050), pantalla OLED y servomotor, gobernado por un solo ESP32 y apoyado en una tubería de visión por computadora (ESP32-CAM) que corría sobre MQTT y Firebase. La implementación final reorientó el proyecto hacia una plataforma de autonomía y multinodo. Los cambios de fondo se resumen así:

Aspecto	Propuesta inicial	Implementación final
Cómputo	1× ESP32 centralizado	3× ESP32 distribuidos (Cerebro S3, Ejecución, Expresión)
Comunicación interna	MQTT (broker + Internet)	ESP-NOW (radio) + UART (cable), sin broker
Nube	Firebase Realtime DB	Supabase (PostgreSQL + REST + Realtime)
"Inteligencia"	Visión por computadora (ESP32-CAM, MediaPipe)	Autonomía reactiva embebida (ISR + máquinas de estado)
Actuación	Servo + OLED + buzzer	2× Motor 550 con ESC + ventilador holográfico
Control humano	No contemplado	Receptor RC con prioridad sobre el modo autónomo

Este reporte documenta el sistema tal como quedó construido. Donde la rúbrica pide un pilar de MQTT o de IA, se documenta el equivalente realmente implementado (malla ESP-NOW/UART y autonomía reactiva, respectivamente), señalando explícitamente la equivalencia.

2. Materiales y equipos

La siguiente lista corresponde a los componentes que realmente quedaron integrados en el prototipo final, conforme al diagrama eléctrico de la sección 4.

Componente	Cantidad	Función en el robot
ESP32-S3-WROOM-1	1	Nodo Cerebro. Único con acceso a Internet; sondea la nube, coordina a los esclavos y vigila el sensor de caída.
ESP32 DevKit V1 (38 pines)	1	Nodo Ejecución. Genera el PWM de los dos ESC y lee el receptor RC.
ESP32 DevKit V1 (30 pines)	1	Nodo Expresión. Controla el ventilador holográfico por TCP/WiFi.
Motor 550	2	Tracción del robot (rueda izquierda y derecha).
ESC QuicRun 880 Brushed	2	Driver de potencia: traduce el PWM en velocidad y sentido de cada motor.
Receptor RC R8EF (8 canales)	1	Control manual del piloto; conserva prioridad sobre el modo autónomo.
Batería LiPo 14.8 V 4S (LP103665)	1	Fuente primaria de energía del robot.
Reductor DC-DC LM2596	2	Convierte 14.8 V a 12 V (ventilador) y a 5 V (lógica).
Power bank	1	Alimentación dedicada del Cerebro, aislada del ruido de los motores.
Módulo óptico TCRT5000	1	Sensor de caída: detecta la pérdida de suelo para el paro de emergencia.
Ventilador holográfico 12 V	1	Proyecta las caras y animaciones (expresión del robot).
Proyecto Supabase	1	Backend en la nube: base de datos, API REST y Realtime.

2.1 Conexionado por nodo (pinout)

Resumen de los pines empleados en cada microcontrolador, tomados del firmware y del diagrama de conexión:

Nodo Cerebro (ESP32-S3)

Pin / recurso	Conexión y propósito
GPIO 4	Señal del módulo TCRT5000 (sensor de caída), con interrupción por cambio.
GPIO 18 (RX1)	Recepción UART desde el TX del nodo Expresión.
GPIO 17 (TX1)	Transmisión UART hacia el RX del nodo Expresión.
WiFi (STA)	Conexión a Internet para hablar con Supabase (REST).
ESP-NOW	Radio hacia el nodo Ejecución (mismo canal que el WiFi).
Alimentación	5 V desde power bank dedicado.

Nodo Ejecución (ESP32 DevKit V1)

Pin / recurso	Conexión y propósito
GPIO 34	Entrada PWM del canal 1 del receptor RC.
GPIO 35	Entrada PWM del canal 2 del receptor RC.
GPIO 25	Salida de señal hacia el ESC del motor 1.
GPIO 26	Salida de señal hacia el ESC del motor 2.
ESP-NOW	Radio hacia el Cerebro (recibe órdenes, devuelve ACK).
Alimentación	5 V desde reductor LM2596.

Nodo Expresión (ESP32 DevKit V1)

Pin / recurso	Conexión y propósito
GPIO 16 (RX2)	Recepción UART desde el TX del Cerebro.
GPIO 17 (TX2)	Transmisión UART hacia el RX del Cerebro.
WiFi (STA)	Conexión a la red propia del ventilador holográfico (MAC clonada).
Alimentación	5 V desde reductor LM2596.

3. Marco teórico

3.1 Microcontroladores ESP32 y ESP32-S3

El ESP32 es un microcontrolador de doble núcleo con WiFi y Bluetooth integrados, muy usado en IoT por su bajo costo y su amplio soporte en el entorno Arduino. El ESP32-S3 es una variante más reciente, con mejor desempeño y más memoria, elegida para el Cerebro porque concentra las tareas más pesadas (red, JSON, coordinación). Los nodos esclavos usan ESP32 DevKit V1 estándar, suficientes para tareas dedicadas (motores y holograma).

3.2 Capa de Abstracción de Hardware (HAL)

Una HAL es una capa de software que oculta los detalles del hardware (pines, registros, temporizadores) detrás de funciones de alto nivel. Su ventaja es el desacoplamiento: la lógica de control pide “mover motores” o “mostrar una cara” sin conocer los microsegundos de PWM ni el protocolo TCP del ventilador. En GARRA-OS cada nodo es, en sí mismo, una HAL especializada; además, todos comparten una misma “interfaz de comunicación” (la estructura `msg_garra`), que actúa como contrato común entre subsistemas.

3.3 PWM, ESC y lectura de receptores RC

Los motores no se conectan directamente al microcontrolador: lo hacen a través de un ESC (Electronic Speed Controller), que recibe una señal de modulación por ancho de pulso (PWM) de 50 Hz, igual que un servomotor. Un pulso de 1500 μ s significa “detenido”, valores menores significan un sentido y mayores el contrario. El mismo lenguaje se usa para leer el receptor RC: la función `pulseIn()` mide cuántos microsegundos dura el pulso que envía el receptor en cada canal, y de ahí se deduce la intención del piloto.

3.4 Comunicación entre dispositivos: ESP-NOW y UART (en lugar de MQTT)

MQTT es un protocolo de publicación/suscripción sobre TCP que organiza los mensajes en “tópicos”, pero requiere un broker y conexión a Internet. En la versión de combate se priorizó la latencia y la robustez, por lo que la mensajería interna se resolvió sin broker mediante dos mecanismos complementarios:

- **ESP-NOW:** protocolo de radio punto a punto de Espressif que entrega tramas cortas en milisegundos sin router. Se usa entre el Cerebro y el nodo de Motores. Requiere que ambos extremos estén en el mismo canal de radio.
- **UART:** comunicación serie por cable (dos hilos, TX y RX) entre el Cerebro y el nodo de Expresión. Es simple, determinista y no depende de la radio.

Ambos transportan la misma estructura de mensaje (tipo, destino, acción), que cumple el papel que en MQTT tendrían los tópicos: el campo “destino” (MOT/EXP/ALL) equivale a la dirección de un tópico, y el campo “acción” al contenido publicado.

3.5 Persistencia en la nube y tiempo real: Supabase (en lugar de Firebase)

Supabase es una plataforma Backend as a Service construida sobre PostgreSQL, que sustituye a Firebase en este proyecto. Aporta tres servicios clave: una base de datos relacional, una API

REST automática (que el ESP32 consume por HTTP, sin SDK) y Realtime, que propaga al instante los cambios de una tabla hacia los clientes suscritos. Dos conceptos son centrales:

- **Polling vs. Realtime:** el ESP32 consulta los comandos por sondeo periódico (polling HTTP GET), porque es un cliente ligero; el dashboard, en cambio, recibe la telemetría “empujada” por Realtime, sin recargar.
- **Row Level Security (RLS):** políticas declarativas que definen, fila por fila, quién puede leer o escribir. En este prototipo de laboratorio se otorga acceso total al rol anónimo, lo que permite usar la llave pública directamente en el firmware y el navegador.

3.6 Autonomía reactiva embebida (en lugar de visión por computadora)

En vez de una pipeline de IA por visión, la inteligencia del robot es reactiva y distribuida. Cada nodo ejecuta su propia máquina de estados finitos (FSM) y reacciona a eventos locales. Dos herramientas de bajo nivel lo hacen posible: las interrupciones por hardware (ISR), rutinas que el procesador ejecuta de inmediato al ocurrir un evento físico (como el cambio del sensor de caída), y las propias FSM, que modelan el comportamiento como un conjunto de estados y transiciones bien definidas. El resultado es un control determinista y predecible, propio de la robótica embebida, distinto del aprendizaje automático.

4. Diagrama del sistema

El siguiente esquema eléctrico muestra las conexiones finales del prototipo: los tres ESP32, la electrónica de potencia (ESC y motores 550), el receptor RC R8EF, la cadena de energía (LiPo 4S → reductores DC-DC → cargas de 12 V y 5 V) y el sensor de caída TCRT5000 sobre el nodo Cerebro.

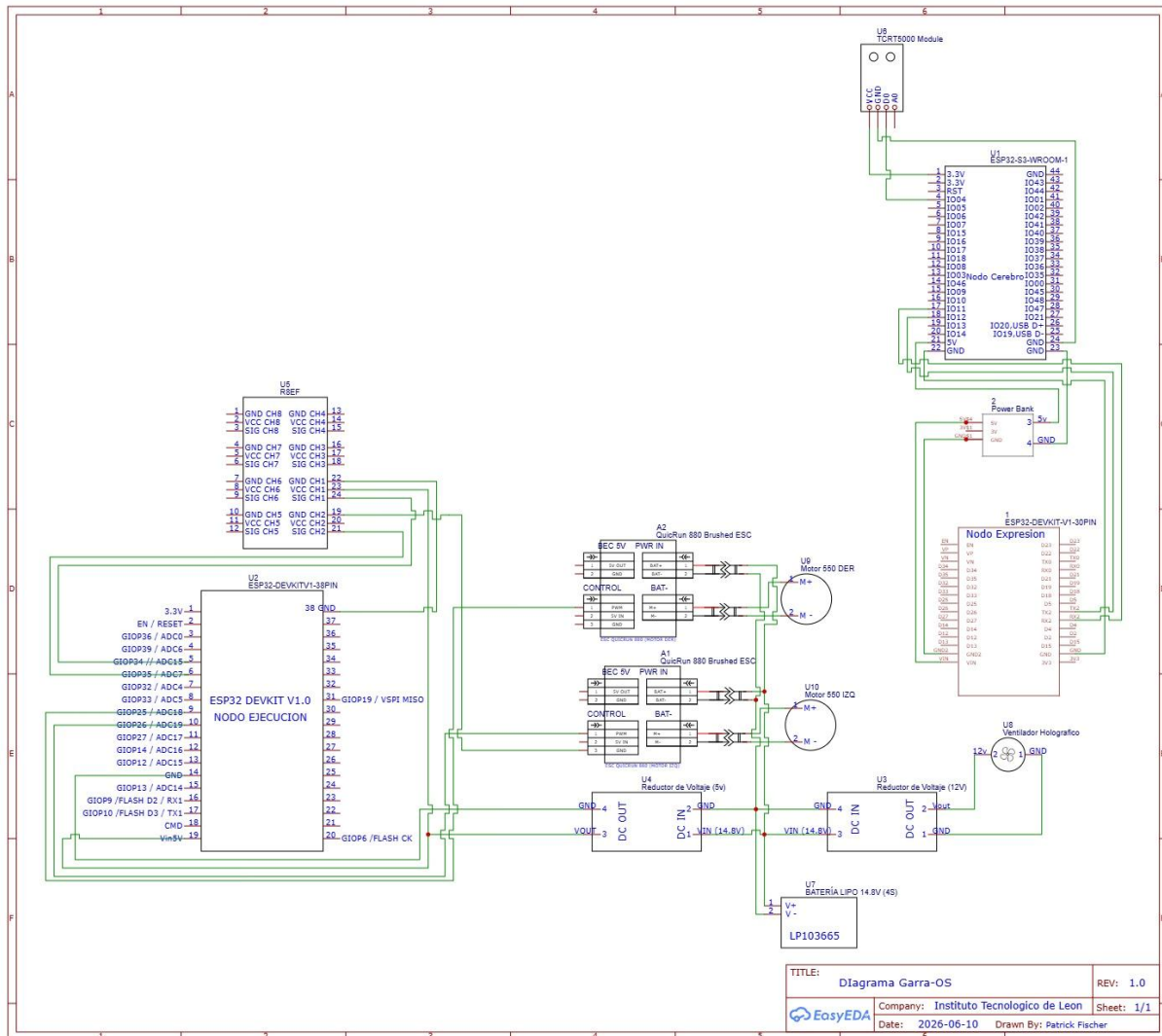


Figura 1. Diagrama de conexión de Chappie (GARRA-OS).

4.1 Arquitectura de datos (Nube / Autonomía)

El sistema se organiza en tres capas (interfaz, servidor/nube y dispositivo) más una malla interna de tres nodos. El flujo completo de extremo a extremo es:

Flujo de datos de extremo a extremo:

```
INTERFAZ WEB --UPDATE--> SUPABASE (control_comandos) --GET polling--> CEREbro (ESP32-S3)
INTERFAZ WEB <--Realtime-- SUPABASE (telemetry_debug) <--POST----- CEREbro

CEREbro --ESP-NOW (radio)--> NODO EJECUCION (2x ESC + Motores 550 + RC R8EF)
CEREbro --UART (cable)-----> NODO EXPRESION (Ventilador Holografico via TCP)

TCRT5000 (interrupcion) --> ALTOTOTAL --> [Motores: neutral 30 s] + [Expresion: cuarentena 30 s]
```

4.2 Descripción de los bloques

- **Nodo Cerebro (ESP32-S3):** orquestador. Es el único conectado a Internet. Sondea Supabase, detecta comandos nuevos, los reparte por radio y por cable, recoge las confirmaciones (ACK) y sube la telemetría. También aloja el reflejo de emergencia.
- **Nodo Ejecución (ESP32):** planta motriz. Lee el receptor RC y controla los dos ESC. Su máquina de estados combina control manual (prioritario), una auto-prueba de giro y la coreografía de baile.
- **Nodo Expresión (ESP32):** personalidad. Traduce las órdenes en “caras” y animaciones proyectadas por el ventilador holográfico, mediante un protocolo TCP propietario.
- **Supabase:** servidor en la nube. Almacena el comando vigente y el historial de telemetría, y empuja los cambios al dashboard.
- **Interfaz web:** puesto de mando del operador. Escribe comandos y visualiza el estado de los tres nodos en vivo.

5. Roles del equipo

La siguiente distribución resume la responsabilidad principal de cada integrante. **Verifiquen y ajusten los nombres por subsistema según el trabajo real antes de la entrega individual.**

Integrante	Responsabilidad principal
Fischer González Patrick	Arquitectura de firmware y electrónica: programación de los tres nodos ESP32, diseño del diagrama de conexión, cadena de potencia, protocolo de mensajes msg_garra e ingeniería inversa del protocolo del ventilador.
Casas Bastidas José Iván	Backend en la nube: modelado de las tablas de Supabase, función y trigger de timestamp, políticas RLS, configuración de Realtime e integración del ciclo comando/telemetría.
Bahena Mora Emilio Salvador	Interfaz y telemetría: dashboard web, suscripción Realtime, lógica asíncrona de envío de comandos, manejo de estados de botón y visualización de los nodos.
Alcalá Ramos Luz Estefanía	Integración mecánica y energía: ensamble del chasis, ruteo del arnés, reductores DC-DC, montaje del sensor de caída y pruebas de campo del prototipo.

6. Desarrollo

El desarrollo se organiza en los cuatro pilares solicitados por la materia, adaptados a la implementación final, más dos apartados de soporte (protocolo del holograma y generación de la coreografía) y la evolución por versiones.

6.1 Pilar 1 — Biblioteca de Control de Hardware (HAL)

La HAL se materializa en tres frentes: el control de motores (nodo Ejecución), la proyección de expresiones (nodo Expresión) y la lectura del sensor de caída (nodo Cerebro).

Control de motores

El nodo de Ejecución encapsula los motores tras la librería ESP32Servo, que produce la señal de 50 Hz que entienden los ESC. La inicialización reserva temporizadores y fija el rango de pulso 1000–2000 µs:

nodo_ejecucion.ino → configuración de los ESC

```
ESP32PWM::allocateTimer(0); ESP32PWM::allocateTimer(1);
esc_ch1.setPeriodHertz(50); esc_ch2.setPeriodHertz(50);
esc_ch1.attach(PIN_ESC_CH1, 1000, 2000);
esc_ch2.attach(PIN_ESC_CH2, 1000, 2000);
```

La lectura del receptor RC se hace con pulseIn() y se valida por rango para descartar lecturas espurias cuando la radio está apagada. El operador se considera “activo” solo si rebasa una zona muerta (DEADBAND), evitando que el ruido del stick cancele el modo autónomo:

nodo_ejecucion.ino → lectura RC y prioridad manual

```
unsigned long raw_ch1 = pulseIn(PIN_CH1, HIGH, 25000);
uint16_t ch1 = (raw_ch1 > 800 && raw_ch1 < 2200) ? raw_ch1 : PWM_NEUTRAL;
bool operador_activo = (abs(ch1 - PWM_NEUTRAL) > DEADBAND) ||
                      (abs(ch2 - PWM_NEUTRAL) > DEADBAND);
if (operador_activo) { estado_actual = MODO_MANUAL; } // prioridad del piloto
```

Proyección de expresiones

El nodo de Expresión encapsula un protocolo TCP propietario (descrito en 6.5) en dos funciones de alto nivel, proyectarHolograma(indice) y alternarEnergiaHolograma(), de modo que el resto del firmware solo decide qué cara mostrar (por ejemplo 0x05 = FELIZ) sin conocer los bytes del protocolo.

Sensor de caída

El Cerebro abstrae el sensor TCRT5000 detrás de una interrupción (ISR) mínima y un filtro anti-rebote, exponiendo a la lógica un único evento de alto nivel: “se perdió el suelo”.

6.2 Pilar 2 — Integración total de la malla de comunicación (ESP-NOW ↔ UART)

En lugar de tópicos MQTT, el Cerebro reparte una estructura de mensaje común a los dos esclavos. La estructura es idéntica byte a byte en los tres nodos, condición indispensable para que ESP-NOW interprete bien las tramas:

Estructura compartida msg_garra

```
typedef struct msg_garra {
    uint8_t tipo; // 1=BOOT 2=ACTION 3=ACK 4=EMERGENCY
    char destino[4]; // "MOT", "EXP", "ALL", "CER"
    char accion[15]; // texto del comando (max 14 + nulo)
} msg_garra;
```

Secuencia de arranque (handshake / “POST”)

Antes de operar, el Cerebro ejecuta una secuencia de arranque que valida toda la red; si algún eslabón falla, se detiene de forma segura y reporta el código de error a la nube:

1. Conecta al WiFi y memoriza el canal de radio (clave para ESP-NOW).
2. Saluda por UART al nodo Expresión y espera su confirmación “3,CER,ACK_EXP” (timeout 10 s).
3. Inicializa ESP-NOW, registra al nodo Motores y le envía BOOT; espera el “ACK_MOT”.
4. Lee el comando inicial en Supabase para sincronizarse con el estado de la nube.

La “tabla de tópicos” del sistema —el repertorio de acciones que viajan por la malla— es la siguiente:

Acción	Destino	Efecto en el robot
BAILAR	ALL	Motores ejecutan la coreografía pregrabada (17 s); Expresión proyecta el holograma de baile (0x00).
SEGUIR	EXP	Rutina de 20 s en dos fases: IRONCLAW (0x06) → RUGIDO (0x07).
ALERTA	EXP	Proyecta la cara de alerta/enojo (0x02).
REPOSO / IDLE	ALL	Vuelve al estado de reposo (cara feliz 0x05).
ALTOTOTAL	ALL	Paro de emergencia: motores a neutral y cuarentena de 30 s.
BOOT	MOT/EXP	Saludo de arranque para validar el enlace.
ACK_MOT / ACK_EXP	CER	Confirmaciones de los esclavos hacia el Cerebro.

El reparto de un comando nuevo se hace simultáneamente por los dos medios físicos:

nodo_cerebro.ino → reparto dual del comando

```
esp_now_send(slaveMAC1, (uint8_t *)&msg, sizeof(msg)); // -> Motores (radio)
Serial1.printf("%d,EXP,%s\n", msg.tipo, msg.accion); // -> Expresion (cable)
```

6.3 Pilar 3 — Autonomía reactiva: el reflejo de emergencia (en lugar de IA de visión)

El comportamiento “inteligente” más crítico es el reflejo de caída. El sensor TCRT5000 dispara una interrupción que solo levanta una bandera; el trabajo pesado se hace en el bucle, con un anti-rebote de 10 ms para descartar falsas alarmas. Si el peligro persiste, se dispara ALTOTOTAL a todos los nodos:

nodo_cerebro.ino → reflejo de emergencia

```
void IRAM_ATTR isrFrenoCaída() { banderaEmergencia = true; } // ISR minima

if (banderaEmergencia) {
  banderaEmergencia = false; delay(10); // anti-rebote optico
  if (digitalRead(PIN_SENSOR_MH) == ESTADO PELIGRO) {
    msg_garra msg_em = {4, "ALL", "ALTOTOTAL"};
    esp_now_send(slaveMAC1, (uint8_t*)&msg_em, sizeof(msg_em)); // -> Motores
    Serial1.print("4,ALL,ALTOTOTAL\n"); // -> Expresion
    postTelemetry(99, "EMERGENCIA_ALTOTOTAL_SENSOR_CAIDA", ...);
  }
}
```

Jerarquía de prioridades

El sistema se diseñó alrededor de una jerarquía estricta que garantiza un comportamiento predecible y seguro:

- **Nivel 1 — Emergencia (ALTOTOTAL):** vence a todo. En el nodo de Motores actúa como cortafuegos que fuerza neutral durante 30 s e ignora cualquier otra orden.
- **Nivel 2 — Control manual del piloto:** vence al modo autónomo. Si el piloto toca los sticks, se aborta cualquier rutina.
- **Nivel 3 — Modo autónomo:** rutinas (BAILAR, SEGUIR) y auto-prueba de giro, solo cuando no hay emergencia ni piloto activo.

Máquinas de estado por nodo

Cada esclavo modela su comportamiento como una FSM. El nodo de Motores transita entre MODO_MANUAL, MACRO_MEDIA_VUELTA y RUTINA_BAILE; el de Expresión entre MODO_IDLE, MODO_BAILE, MODO_SEGUIR y MODO_CUARENTENA. Esta separación evita condicionales enredados y hace explícito qué puede ocurrir en cada momento.

6.4 Pilar 4 — Persistencia en la nube y dashboard (Supabase ↔ ESP32 ↔ Interfaz)

El backend se define con un único script SQL que crea dos tablas de roles opuestos, sus políticas de seguridad y la configuración de Realtime.

Tabla	Patrón de diseño	Rol en el sistema
control_comandos	Fila única (id=1)	Flujo Web → ESP32. Guarda el comando vigente; el Cerebro la sondea por GET.
telemetria_debug	Bitácora (append-only)	Flujo ESP32 → Web. Historial de estados; dispara eventos Realtime al dashboard.

The diagram shows two table schemas. The first table, **control_comandos**, has four columns: **id** (int4, primary key), **comando** (varchar), **parametros** (jsonb), and **updated_at** (timestampz). The second table, **telemetria_debug**, has eight columns: **id** (uuid, primary key), **estado_maestro** (varchar), **estado_motores** (varchar), **estado_expresion** (varchar), **codigo_error** (int4), **mensaje** (text), and **updated_at** (timestampz).

Figura 2. Esquema de las tablas en Supabase.

The screenshot shows the Supabase data browser for the **control_comandos** table. It displays a single row with the following data: **id** is 1, **comando** is BAILAR, **parametros** is an empty object {}, and **updated_at** is 2026-06-11 06:46:34.650174+00. The interface includes a search bar, a sort button, a status indicator for RLS (disabled), a role selector (postgres), and an insert button.

Figura 3. Fila única de control_comandos con el comando vigente (“BAILAR”).

Un trigger mantiene actualizado el campo **updated_at** en cada cambio de **control_comandos**, lo que permite al Cerebro saber si un comando es nuevo. Las políticas RLS otorgan acceso total al rol anónimo (entorno de laboratorio):

schema_supabase.sql —> trigger, RLS y Realtime

```
CREATE TRIGGER update_control_comandos_modtime
BEFORE UPDATE ON public.control_comandos
FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();

ALTER TABLE public.control_comandos ENABLE ROW LEVEL SECURITY;
CREATE POLICY "acceso anonimo" ON public.control_comandos
FOR ALL TO anon USING (true) WITH CHECK (true);

ALTER PUBLICATION supabase_realtime ADD TABLE public.telemetria_debug;
```

Del lado del ESP32, la lectura del comando es deliberadamente ligera (pide solo una columna de una fila); el dashboard escribe con un **UPDATE** y escucha la telemetría por Realtime:

schema_supabase.sql / nodo_cerebro.ino / index.html

```
// ESP32: lectura ligera del comando vigente
http.begin(SUPABASE_URL + "/rest/v1/control_comandos?id=eq.1&select=comando");
if (http.GET() == HTTP_CODE_OK) { /* parseo JSON */ }

// Dashboard: envio de un comando (Web -> ESP32)
await db.from('control_comandos').update({ comando: 'BAILAR' }).eq('id', 1);

// Dashboard: escucha en vivo (ESP32 -> Web)
db.channel('cambios').on('postgres_changes',
  { event: '*', schema: 'public', table: 'telemetria_debug' },
  (p) => actualizarConsolaDOM(p.new)).subscribe();
```

6.5 Desglose del protocolo del ventilador holográfico

El ventilador holográfico es un producto comercial cerrado: emite su propia red WiFi y solo acepta órdenes de una MAC concreta, por lo que el nodo de Expresión clona esa MAC. Mediante ingeniería inversa se identificaron cuatro tipos de paquete TCP, todos enmarcados por las MAC de origen y destino en formato ASCII. Para proyectar una imagen se envían tres paquetes en secuencia (KNOCK → SELECT → PLAY):

Paquete	Tamaño	Contenido / propósito
KNOCK	24 bytes	Handshake inicial (MAC origen + MAC destino).
POWER	28 bytes	Bytes 'c','c','a' → alterna encendido/apagado del ventilador.
SELECT	29 bytes	Bytes 'c','d','B' + índice de imagen → elige la cara/animación.
PLAY	32 bytes	Bytes 'c','g','b' + n° de secuencia → fuerza la reproducción.

El catálogo de imágenes mapeado a comandos del robot es:

Índice	Expresión / uso
0x00	BAILE (animación de baile)
0x02	ENOJADO (usado como ALERTA)
0x05	FELIZ (estado de reposo / IDLE)
0x06	IRONCLAW (primera fase de SEGUIR)
0x07	RUGIDO (segunda fase de SEGUIR)

6.6 Generación de la coreografía (herramienta Grabadora)

La rutina de baile no se programó “a mano”: se grabó. Un firmware auxiliar (grabadora_macros.ino) se carga temporalmente en el nodo de Motores y, mientras el piloto mueve los sticks, mide los pulsos del receptor por interrupciones y los imprime cada 100 ms (10 Hz) en el formato exacto del arreglo del firmware final. Esa lista de pares { ch1, ch2 } se copia y pega en la constante MACRO_BAILE de nodo_ejecucion.ino, que luego se reproduce de forma proporcional al tiempo y atenuada al 35 % de la potencia, por seguridad en la demostración.

grabadora_macros.ino + nodo_ejecucion.ino

```
// La grabadora imprime cada 100 ms una 'foto' de los dos sticks:
Serial.printf("  {%u, %u},\n", c1, c2);  // -> se pega en MACRO_BAILE[][2]

// En el firmware final se reproduce el frame que toca a cada milisegundo,
// atenuando la energia (centrada en 1500 us) al 35%:
int frame = (tiempo_transcurrido * ELEMENTOS_BAILE) / DURACION_BAILE;
uint16_t pwm = 1500 + ((MACRO_BAILE[frame][0] - 1500) * 35 / 100);
```

6.7 Evolución del firmware por versiones

El desarrollo fue incremental; cada nodo pasó por varias versiones hasta su forma final. El repositorio conserva únicamente las versiones definitivas, pero la evolución documenta el proceso:

Nodo	Versión final	Capacidades añadidas a lo largo del desarrollo
Cerebro	v3	Base de producción: WiFi + Supabase + ESP-NOW + UART; se añadió el reflejo de emergencia por sensor TCRT5000 y la FSM de sondeo/espera.
Ejecución	v4	De control híbrido (manual + ESP-NOW) a v3 con rutina SEGUIR y, finalmente, v4 con la coreografía BAILE pregrabada y el cortafuegos ALTOTOTAL.
Expresión	v4	De “modo simulación” inicial a puente UART↔TCP real con toggle de energía, cuarentena y macro SEGUIR; v4 (“dieta”) optimizó memoria y estabilidad.

7. Retos técnicos y soluciones

Durante la integración surgieron problemas representativos de los sistemas embebidos distribuidos. La siguiente tabla resume los principales y cómo se resolvieron:

Reto	Causa	Solución aplicada
ESP-NOW no entregaba tramas	Emisor y receptor en canales de radio distintos	El Cerebro lee su canal WiFi; los esclavos escanean la red "honor" y fijan el mismo canal.
Falsas alarmas del sensor de caída	Rebote óptico/eléctrico del comparador	ISR mínima + reconfirmación con anti-rebote de 10 ms en el loop.
Latencia / saturación con la nube	Polling HTTP demasiado frecuente o pesado	Tabla de fila única y GET de una sola columna; sondeo cada ~1 s.
Envío doble de comandos	Pulsaciones rápidas en el dashboard	Bloqueo por estado del botón (sending/sent) mientras la promesa está en curso.
Reinicios por caídas de tensión	Picos de corriente de los motores en el bus de 5 V	Reductores DC-DC independientes y power bank dedicado al Cerebro.
Pérdida de control en emergencia	Órdenes remotas compitiendo con el paro	Cortafuegos ALTOTOTAL con prioridad absoluta y cuarentena de 30 s.

8. Resultados

Video demostrativo:

https://drive.google.com/file/d/1EL903J_0tb7EZA_NgJykTLE4_yCpa2Bh/view?usp=sharing

Repositorio en GitHub:

<https://github.com/Ivan-Casas/GARRA-OS-Reporte-Final-Repositorio-y-Video>

El prototipo final quedó encapsulado en su chasis con todos los subsistemas integrados y operando de extremo a extremo: el operador envía un comando desde el dashboard, el Cerebro lo sondea de Supabase y lo reparte por ESP-NOW/UART, los motores y el holograma reaccionan, y la telemetría regresa en tiempo real a la interfaz. El reflejo de emergencia por sensor de caída se verificó levantando el robot del suelo, comprobando que los motores se detienen y el ventilador entra en cuarentena.

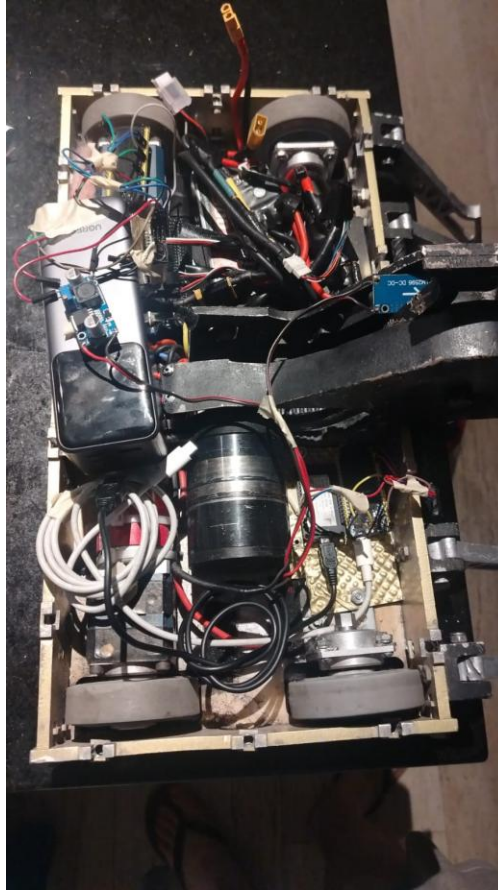


Figura 4. Vista superior del prototipo integrado: motores 550, batería LiPo, reductores y nodos.

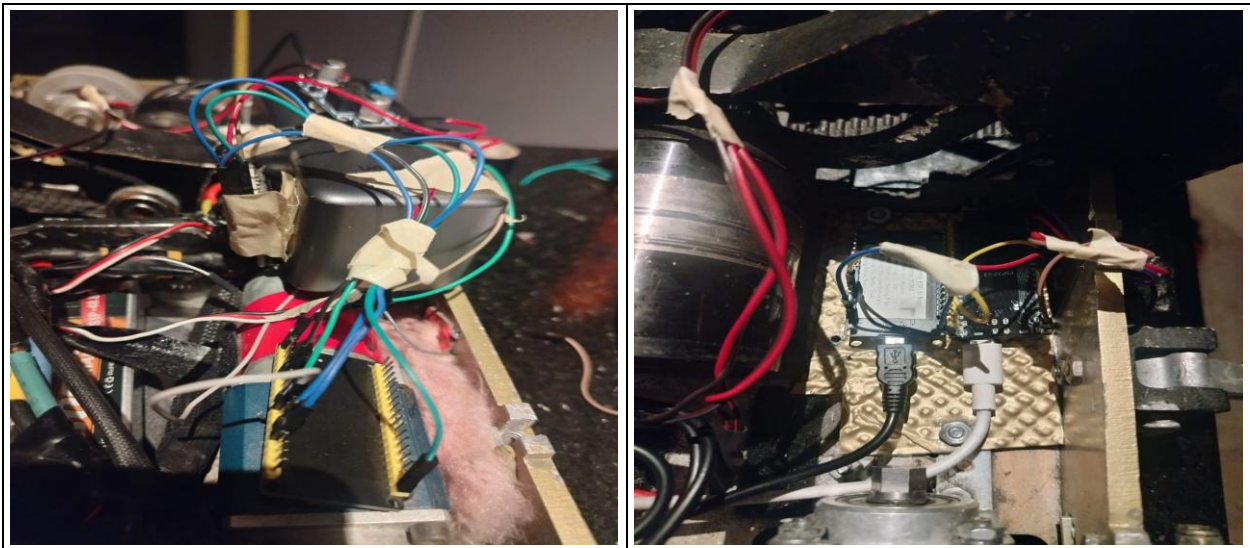


Figura 5. Cableado interno: nodos ESP32, motor de tracción y arnés de potencia.

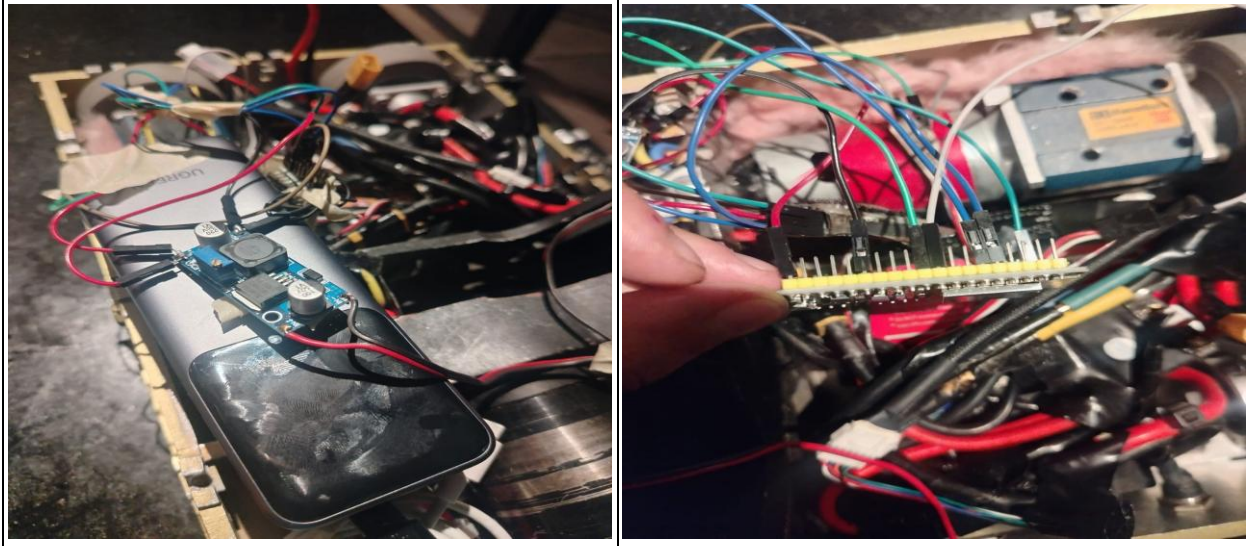


Figura 6. Detalle del reductor DC-DC y del conexionado de los nodos esclavos.

9. Conclusiones Grupales

Con GARRA-OS aprendimos que un robot funciona mejor cuando el trabajo se reparte entre varios cerebros pequeños en lugar de cargárselo todo a uno solo. Al darle a cada parte (los motores, la cara y el coordinador) su propia tarea, el robot se volvió más estable: si una parte fallaba, las demás seguían funcionando.

También fue clave la forma en que se comunican. En lugar de depender de Internet para todo, el robot reacciona por su cuenta de manera casi instantánea y la conexión a Internet se usa solo para mandarle órdenes y ver cómo está, que es justo donde no importa si tarda un poquito. Separar lo que tiene que ser inmediato de "lo que puede esperar" fue la mejor decisión que tomamos.

Otro gran aprendizaje fue poner reglas claras de quién manda: el piloto siempre tiene prioridad sobre el modo automático, y el botón de emergencia gana por encima de todo. Eso hizo que el robot fuera predecible y seguro, incluso cuando algo salía mal. Además nos quedó claro que el cableado y la energía son tan importantes como la programación: varios problemas que parecían errores de código en realidad eran fallas de corriente.

Para mejorar a futuro nos gustaría ordenar y fijar bien el cableado, guardar las contraseñas en un archivo aparte y reforzar la seguridad si el robot se usara fuera del laboratorio. En resumen, el equipo logró entregar el proyecto completo: el robot físico funcionando, el código bien organizado y esta documentación.

10. Análisis individual por integrante

Fischer González Patrick (23240045)

Rol detallado: Programé los tres cerebros del robot (los tres ESP32), definí la forma en que se mandan mensajes entre ellos, hice el diagrama de conexiones, armé la parte de energía y potencia, y descubrí cómo darle órdenes al ventilador holográfico.

Problema técnico enfrentado: Lo más difícil fue lograr que el cerebro principal y el de los motores se entendieran por radio: los dos tienen que estar en el mismo canal, y ese canal depende de la red WiFi. Lo resolví haciendo que el cerebro detectara su canal y que los demás se ajustaran al mismo. El segundo reto fue descifrar cómo hablarle al ventilador, que solo hacía caso después de un saludo especial.

Conclusión personal: Aprendí que en un sistema con varias piezas, la forma en que se conectan importa tanto como el programa: si dos partes no se escuchan, de nada sirve que el código sea perfecto. También me quedó claro lo valioso que es tener un botón de emergencia que mande por encima de todo y dejar que cada parte se encargue de su propia tarea.

Casas Bastidas José Iván (23240883)

Rol detallado: Armé la parte en la nube, las dos tablas donde se guardan las órdenes y el estado del robot, y la configuración para que la página web reciba la información al instante y de forma segura.

Problema técnico enfrentado: El reto fue el tiempo de respuesta: el robot revisa la nube cada cierto tiempo para ver si hay órdenes nuevas; si revisaba muy seguido se saturaba, y si lo hacía muy despacio el robot tardaba en reaccionar. Lo resolví dejando una sola casilla con la orden actual y pidiendo únicamente el dato necesario, para que la consulta fuera ligera y el robot pudiera revisar cada segundo sin problemas.

Conclusión personal: Entendí la diferencia entre guardar el estado actual (una sola casilla que se actualiza) y guardar un historial (que va sumando registros), y cuándo conviene cada uno. También aprendí a configurar los permisos de seguridad de una base de datos en la nube.

Bahena Mora Emilio Salvador (23240009)

Rol detallado: Hice la página web de control: los botones para mandar órdenes, la pantalla que muestra en vivo el estado de las tres partes del robot y los avisos de error.

Problema técnico enfrentado: Mi reto fue evitar que se mandaran órdenes repetidas: al picar rápido un botón se enviaban varias veces. Lo resolví bloqueando el botón mientras la orden se procesaba y mostrando un aviso que confirma si llegó o si hubo un error. Además, la pantalla se pone roja cuando el robot reporta una falla.

Conclusión personal: Aprendí a hacer que una página web se comunique con un servicio real y a diseñar una pantalla que le avisa claramente al usuario qué está pasando, sin que tenga que

adivinar si su orden funcionó. Me di cuenta de que los avisos visuales son tan importantes como la función en sí.

Alcalá Ramos Luz Estefanía (23240079)

Rol detallado: Me encargué del armado físico y de la energía: montar el chasis, acomodar los cables, instalar los reductores de voltaje, colocar el sensor de caída y hacer las pruebas con el robot.

Problema técnico enfrentado: El reto fue la energía: la batería entrega un voltaje alto, pero el ventilador y la electrónica necesitan voltajes más bajos, y cuando los motores jalaban mucha corriente el robot se reiniciaba. Lo resolví usando reductores de voltaje separados y alimentando el cerebro con una batería aparte, para que el "ruido" eléctrico de los motores no afectara a la electrónica.

Conclusión personal: Entendí que la parte eléctrica y el orden de los cables son tan importantes como la programación: un problema de corriente puede tirar todo el sistema aunque el programa esté perfecto. También aprendí a colocar y orientar bien un sensor para que mida de forma confiable.

11. Lista de verificación de entrega

Control de los requisitos de la rúbrica antes de subir la entrega:

Requisito	Puntos	Estado
Repositorio con jerarquía HAL / Servidor / Interfaz	E7	Hecho
README con instalación y ejecución	E7	Hecho
Encabezado (Objetivo, Integrantes, Proyecto) en cada archivo	-20%	Hecho
Código documentado línea por línea	-20%	Hecho
Reporte con conclusiones y problemas individuales	-40%	Hecho
Diagrama del sistema actualizado	Reporte	Hecho
Fotos del prototipo final	E8/Reporte	Hecho
Enlace al video horizontal (≤ 6 min)	E8	Hecho
Referencias en formato APA	Reporte	Hecho

12. Glosario técnico

Término	Definición breve
HAL	Capa de abstracción de hardware; expone funciones de alto nivel ocultando pines y registros.
ESP-NOW	Protocolo de radio punto a punto de Espressif, sin router ni broker, de baja latencia.
UART	Comunicación serie por cable (TX/RX) entre dos dispositivos.

Término	Definición breve
PWM	Modulación por ancho de pulso; señal usada para controlar ESC y servos (50 Hz).
ESC	Controlador electrónico de velocidad; traduce el PWM en potencia para el motor.
ISR	Rutina de servicio de interrupción; código que se ejecuta de inmediato ante un evento físico.
FSM	Máquina de estados finitos; modelo de comportamiento por estados y transiciones.
Polling	Sondeo periódico de un recurso (aquí, la tabla de comandos en Supabase).
Realtime	Servicio de Supabase que empuja los cambios de una tabla a los clientes suscritos.
RLS	Row Level Security; políticas de acceso por fila en PostgreSQL/Supabase.
Brownout	Caída de tensión que reinicia el microcontrolador.
ALTOTOTAL	Comando de paro de emergencia con prioridad absoluta en GARRA-OS.

13. Referencias

- Espressif Systems. (2024). ESP-NOW User Guide y ESP-IDF Programming Guide. Recuperado de <https://docs.espressif.com>
- Supabase. (2024). Supabase Documentation: Database, REST API, Realtime y Row Level Security. Recuperado de <https://supabase.com/docs>
- OASIS. (2019). MQTT Version 5.0 Specification. OASIS Standard. <https://docs.oasis-open.org/mqtt/mqtt/v5.0/>
- Arduino. (2024). Arduino Reference y ESP32 Arduino Core (WiFi, HTTPClient, ESP32Servo). Recuperado de <https://www.arduino.cc/reference>
- Blanchon, B. (2024). ArduinoJson Documentation (v6). Recuperado de <https://arduinojson.org>
- PostgreSQL Global Development Group. (2024). PostgreSQL 16 Documentation: Triggers and Row Security Policies. <https://www.postgresql.org/docs/>